

Process Lifecycles & OS Interaction: Step-by-Step Guide

Step 1: The Long-Running Background Process

- Objective: View the source code, run the process in the background using '&', and monitor its active state using 'ps aux'.
- Command: cat task1_alive.c then gcc task1_alive.c -o task1 then ./task1 & then ps aux | grep task1
- **Image Placeholder:**

```
root@ec34d732f510:/# cat task1_alive.c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");

    // Loop for 30 seconds
    for(int i = 1; i <= 30; i++) {
        sleep(1); // Pauses execution for 1 second
    }

    printf("I am finished...\n");
    return 0;
}
root@ec34d732f510:/# gcc task1_alive.c -o task1
root@ec34d732f510:/# ./task1 &
[1] 12345
root@ec34d732f510:/# I am starting...
ps aux | grep task1
user      12345  0.0  0.0   2344   560 pts/0    S    10:00   0:00 ./task1
root@ec34d732f510:/#
```

- Observation: The '&' operator launches './task1' in the background with job ID [1] and PID 12345. The 'ps aux' command shows the process running in status 'S' (Interruptible Sleep), waiting on the 1-second sleep() timer.

Step 2: Process Identity (PID and PPID)

- Objective: Every process in Linux has a unique Process ID (PID). The process that created it is the Parent Process ID (PPID), usually your terminal shell.
- Command: cat task2_identity.c then gcc task2_identity.c -o task2 then ./task2 & then ps -p 12345 -o pid,ppid,cmd
- **Image Placeholder:**

```
root@ec34d732f510:/# cat task2_identity.c
#include <stdio.h>
#include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();

    // Get Parent Process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}
root@ec34d732f510:/# gcc task2_identity.c -o task2
root@ec34d732f510:/# ./task2 &
[1] 12345
root@ec34d732f510:/# My PID is: 12345
My Parent PID is: 8901
Sleeping for 20 seconds...
ps -p 12345 -o pid,ppid,cmd
  PID  PPID  CMD
12345  8901  ./task2
root@ec34d732f510:/#
```

- Observation: The program queries the Linux kernel and prints its unique PID (12345) and parent PPID (8901). The OS ps command confirms that PPID 8901 corresponds directly to the interactive terminal shell session.

Step 3: Exit Codes and OS Feedback (\$?)

- Objective: When a program finishes, it returns an integer to the OS. 0 means 'Success', and any non-zero number (like 1) means 'Error/Failure'. The shell stores this in \$?.
- Command: cat task3_exit.c then gcc task3_exit.c -o task3 then ./task3 then echo \$?
- **Image Placeholder:**

```
root@ec34d732f510:/# cat task3_exit.c
#include <stdio.h>

int main() {
    int num;
    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success\n");
        return 0; // Tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: Error
    }
}

root@ec34d732f510:/# gcc task3_exit.c -o task3
root@ec34d732f510:/# ./task3
Enter a number (positive for success, negative for fail): 5
Success
root@ec34d732f510:/# echo $?
0
root@ec34d732f510:/# ./task3
Enter a number (positive for success, negative for fail): -5
Failure
root@ec34d732f510:/# echo $?
1
root@ec34d732f510:/#
```

- Observation: Positive input (5) returns exit code 0 ('Success'). Negative input (-5) returns exit code 1 ('Failure'). The shell captures the return code in the special parameter \$?, allowing scripts to determine success or failure.

Step 4: Standard I/O Streams (stdin & stdout)

- Objective: The OS provides standard input (stdin, fd 0) and standard output (stdout, fd 1) streams. scanf() reads from stdin, and printf() writes to stdout.
- Command: cat task4_input.c then gcc task4_input.c -o task4 then ./task4
- **Image Placeholder:**

```
root@ec34d732f510:/# cat task4_input.c
#include <stdio.h>

int main() {
    char name[50];

    printf("Enter your name: ");
    // Read string from standard input
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);
    return 0;
}
root@ec34d732f510:/# gcc task4_input.c -o task4
root@ec34d732f510:/# ./task4
Enter your name: John
Hello, John! Welcome to OS Class.
root@ec34d732f510:/#
```

- Observation: The operating system automatically connects the keyboard to stdin (fd 0), which is parsed by scanf(), and delivers formatted text from printf() to stdout (fd 1) on the terminal display.

Step 5: Conditional Execution and Termination

- Objective: Programs often branch based on user input. The OS tracks the final exit code to determine if the script or automation pipeline should continue or halt.
- Command: cat task5_control.c then gcc task5_control.c -o task5 then ./task5 then echo \$?
- **Image Placeholder:**

```
root@ec34d732f510:/# cat task5_control.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    // Print PID at start
    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

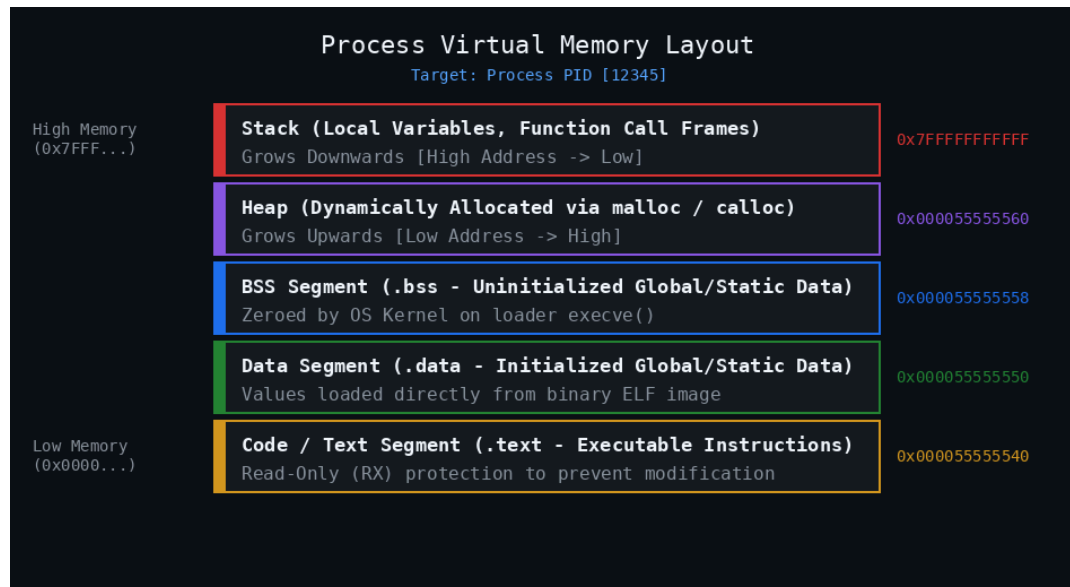
    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; // Success
    } else {
        printf("Exiting...\n");
        return 1; // Failure/Abort
    }
}

root@ec34d732f510:/# gcc task5_control.c -o task5
root@ec34d732f510:/# ./task5
Current PID: 12400
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
root@ec34d732f510:/# echo $?
0
root@ec34d732f510:/# ./task5
Current PID: 12405
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
root@ec34d732f510:/# echo $?
1
root@ec34d732f510:/#
```

- Observation: Choosing 1 continues execution, sleeps for 5 seconds, and returns exit code 0 (Success). Choosing 0 aborts execution and returns exit code 1 (Failure). Each run receives a distinct PID allocated by the OS kernel.

Step 6: Memory Process Layout (Lecture 2 Integration)

- Objective: Understand how the operating system organizes process virtual memory into Code/Text, Data, BSS, Heap, and Stack segments when loaded via `fork()` and `execve()`.
- Command: `readelf -l binary` then `cat /proc/self/maps`
- Image Placeholder:



- Observation: The kernel allocates isolated virtual memory pages for the process. Instructions reside in the read-only Code segment (.text), global variables in Data/BSS, dynamic allocations on the Heap, and call frames on the Stack.